



ค่ายโอลิมปิกวิชาการ ครั้งที่ 18
ศูนย์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

Hashing & Priority Queue

สถิตย์ ประสมพันธ์

Sathit.p@sci.kmutnb.ac.th

Hashing

- ❑ เป็นเทคนิคการค้นหาข้อมูลที่มีประสิทธิภาพสูงมาก
- ❑ แนวคิดของแฮชซึ่งก็คือการจัดเก็บข้อมูลเป็นกลุ่ม (Distribution) ซึ่งจะถูเก็บอยู่ในพื้นที่หน่วยความจำ ที่เรียกว่าตารางแฮช (hash table) และหาวิธีการเข้าถึงคีย์ข้อมูล
- ❑ เทคนิคของแฮชซึ่งจะทำการเข้ารหัสคีย์ข้อมูลเพื่อให้ได้แอดเดรสของตารางแฮชสำหรับการเข้าถึงหรือนำข้อมูลไปเก็บไว้ โดยฟังก์ชันแฮช (hash function) จะทำหน้าที่คำนวณตำแหน่งแอดเดรสดังกล่าว
- ❑ ดังนั้น หากฟังก์ชันแฮชมีความสามารถกระจายคีย์ข้อมูลได้ดี การทำแฮชซึ่งจะเป็นวิธีการที่มีประสิทธิภาพในการค้นหาข้อมูลสูง เนื่องจากมีเปรียบเทียบคีย์ข้อมูลน้อยมาก

ฟังก์ชันแฮช (Hash function)

ฟังก์ชันแฮชที่ดี ต้องมีคุณสมบัติดังนี้

- ต้องมีความสามารถในการกระจายคีย์ข้อมูลได้ดี หมายความว่าคีย์ข้อมูลจะต้องถูกกระจายไปอยู่ในตำแหน่งที่ไม่ซ้ำกันในตารางแฮช
- ต้องง่ายและรวดเร็วในการคำนวณ

ตัวอย่างของฟังก์ชันแฮชที่นิยมใช้งานกันมาก ได้แก่

- ฟังก์ชัน Modulo
- ฟังก์ชัน Mid Square
- ฟังก์ชัน Folding

ฟังก์ชันแฮชแบบ Modulo

กำหนดให้ **k** เป็นคีย์ข้อมูล ตำแหน่งแอดเดรสของ **k** ในตารางแฮชสามารถคำนวณได้ ดังนี้

$$H(k) = k \bmod N + L_0$$

โดยที่ **N** คือขนาดของตารางแฮช และ

L₀ คือแอดเดรสเริ่มต้นในตารางแฮช

ฟังก์ชันแฮชแบบ Modulo

ตัวอย่างเช่น ต้องการนำคีย์ข้อมูล 31 42 51 และ 16 ไปเก็บไว้ในตารางแฮชขนาด 6 ช่อง โดยเริ่มต้นที่แอดเดรสที่ 1 กำหนดให้ฟังก์ชันแฮช คือ $H(k) = k \bmod 6 + 1$

ดังนั้น

$$H(31) = 31 \bmod 6 + 1 = 1 + 1 = 2$$

$$H(42) = 42 \bmod 6 + 1 = 0 + 1 = 1$$

$$H(51) = 51 \bmod 6 + 1 = 3 + 1 = 4$$

$$H(16) = 16 \bmod 6 + 1 = 4 + 1 = 5$$

Hash table

42	1
31	2
	3
51	4
16	5
	6

ฟังก์ชันแฮชแบบ mid square

หลักการของฟังก์ชันแฮชแบบ mid square คือการคำนวณตำแหน่งแอดเดรสของคีย์ข้อมูล โดยแสดงในรูปแบบของไบนารีบิต (binary bits) จากนั้นพยายามค้นหาเลขไบนารีบิตขนาด t บิตที่ไม่ซ้ำกัน เพื่อใช้อ้างอิงตำแหน่งแอดเดรสของตารางแฮช โดยฟังก์ชันแฮชแบบ mid square จะเป็นดังนี้

$$H(k) = \text{mid}(k^2, 2^t)$$

โดยที่ k หมายถึงคีย์ข้อมูล

t หมายถึงจำนวนบิตที่ต้องการ ซึ่ง $n \leq 2^t$

ฟังก์ชันแฮชแบบ mid square

ตัวอย่างเช่น กำหนดให้ตารางแฮชมีขนาด 16 ช่อง ดังนั้นจำนวนบิตที่ต้องการแสดงแอดเดรส คือ $16 \leq 2^t$ ซึ่ง t จะมีค่าเท่ากับ 4 บิต นั่นเอง

K	K ²	binary patterns
31	961	0 0 1 1 1 1 0 0 0 0 0 1
42	1764	0 1 1 0 1 1 1 0 0 1 0 0
51	2601	1 0 0 1 0 1 1 0 0 0 0 1
14	196	0 0 0 0 1 1 0 0 0 1 0 0
12	144	0 0 0 0 1 0 0 1 0 0 0 0
3	9	0 0 0 0 0 0 0 0 1 0 0 1
7	49	0 0 0 0 0 0 1 1 0 0 0 1
17	289	0 0 0 1 0 0 1 0 0 0 0 1

ภาพตัวอย่างการค้นหาตำแหน่งแอดเดรสจากคีย์ข้อมูล โดยวิธีการ mid square

เมื่อได้ตำแหน่งของแอดเดรสของคีย์ข้อมูลในตารางแฮชแล้ว ก็จะนำคีย์ข้อมูลไปเก็บไว้ในตารางแฮช

ฟังก์ชันแฮชแบบ mid square

```
// C++ program to illustrate the
// mid-square hashing technique
#include <ctime>
#include <iostream>

using namespace std;

// Returns a seed value based on current system time.
long long int newTime()
{
    // Acquiring number of seconds
    // passed from Jan 1, 1970.
    time_t t = time(NULL);

    // Converting the time to year, month,
    // day, hour, minute, and second.
    struct tm* tm = localtime(&t);
    long long int x;

    // Applying a certain logic to get
    // required number of digits. It may vary.
    x = (tm->tm_hour) * 10000000 + (tm->tm_min) * 100000
        + (tm->tm_sec) * 1000 + (tm->tm_mday) * 10 + (tm->tm_year);

    // Return the calculated number.
    return x;
}
```



ฟังก์ชันแฮชแบบ mid square

```
// Returns a random 8-digit key.
long int getKey()
{
    // Taking the key from system time.
    // returns a 8-digit seed value.
    static long long int key = newTime();

    // Squaring the key.
    key = key * key;

    // Extracting required number of digits ( here, 8 ).
    if (key < 10000000000000000) {
        key = key / 10000;
        key = key % 1000000000;
    }
    else {
        key = key / 10000;
        key = key % 1000000000;
    }

    // Returning the key value.
    return key;
}
```

```
// Driver Code
int main()
{
    // get the first key
    std::cout << getKey() << endl;

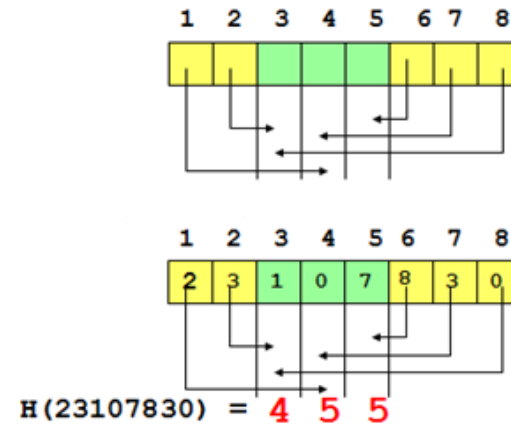
    // get the second key
    std::cout << getKey() << endl;
    return 0;
}
```

ฟังก์ชันแฮชแบบ folding

หลักการของฟังก์ชันแฮชแบบ folding จะมีหลักการคล้ายกับแบบ mid square คือมีการกำหนดหลักของข้อมูลแทนตำแหน่งแอตเดรสในตารางแฮช แต่วิธีการนี้จะกำหนดหลักข้อมูลในรูปแบบของเลขฐานสิบ เช่น จากตัวอย่างด้านล่างหลักที่ใช้เป็นตำแหน่งแอตเดรสในตารางแฮช ได้แก่หลักที่ 3 4 และ 5 ตามลำดับ ขณะที่หลักที่เหลือซึ่งไม่ได้ถูกกำหนดเป็นตำแหน่งแอตเดรสจะต้องนำไปรวมกับหลักดังกล่าว เพื่อถอดรหัสเป็นตำแหน่งแอตเดรสที่แท้จริง

ตัวอย่างเช่น สมมติว่าตารางแฮชมีขนาด 1000 ช่อง ซึ่งจะต้องใช้หลักข้อมูล 3 หลัก เพื่อแสดงตำแหน่งแอตเดรสแต่ละช่องของตารางแฮช ได้แก่ 000 ถึง 999 ในกรณีนี้เราอาจเลือกหลักที่ 3 4 และ 5 จากนั้นหลักที่เหลือ ได้แก่หลักที่ 1 2 6 7 8 จะถูกนำไปรวมกับหลักที่ 3 4 และ 5 สอดคล้องกับตำแหน่งของมัน

ฟังก์ชันแฮชแบบ folding



ภาพการคำนวณตำแหน่งแฮชของคีย์ข้อมูล 23107830 โดยวิธีการพับข้อมูล

การชนกัน (Collision)

ถึงแม้ว่าฟังก์ชันแฮชจะมีความสามารถในการกระจายข้อมูลได้ดีแค่ไหนก็ตาม คีย์ข้อมูลที่มีค่าแตกต่างกันก็ยังมีโอกาสจะได้ตำแหน่งแอดเดรสในตารางแฮชเดียวกัน เราเรียกเหตุการณ์นี้ว่าการชนกัน (collision) สมมติว่า k_1 และ k_2 เป็นคีย์ข้อมูล $h(k_1) = h(k_2)$

นิยามการชนกันของคีย์ข้อมูล กำหนดให้ k_1 และ k_2 เป็นคีย์ข้อมูล การชนกันในตารางแฮชหมายความว่า $h(k_1) = h(k_2)$ โดยที่ $h(.)$ เป็นฟังก์ชันแฮช

วิธีการแก้ปัญหาของการชนกันของคีย์ข้อมูล สามารถทำได้ 2 วิธี

- Open Hashing หรือ Separating chaining
- Closed Hashing หรือ Rehashing

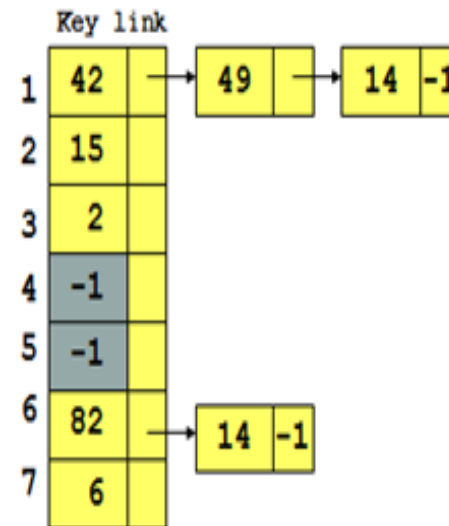
Open Hashing หรือ Separating chaining

เมื่อไรก็ตามที่เกิดการชนกันของคีย์ข้อมูล ให้ทำการขยายพื้นที่ในตารางแฮชเพื่อจัดเก็บคีย์ข้อมูลที่มีตำแหน่งแฮชเดียวกันจากฟังก์ชันแฮช วิธีการนี้ทำได้โดยอาศัยข้อดีของลิสต์เชื่อม ส่วนข้อเสียของวิธีการนี้ก็คือหากคีย์ข้อมูลชนกันมากๆ จะทำให้ลิสต์เชื่อมมีความยาวและเสียเวลาในการเข้าถึงข้อมูลนั่นเอง

keys

42
49
82
14
2
15
5
6

$$\Rightarrow H(K) = K \bmod 7 + 1$$



Closed Hashing หรือ Rehashing

วิธีการนี้จะหลีกเลี่ยงการเพิ่มพื้นที่หน่วยความจำ เมื่อมีการชนกัน แต่จะใช้วิธีคือการคำนวณหาแอดเดรสใหม่ที่ว่างแล้วนำไปคีย์ข้อมูลที่ชนกันเก็บไว้ในตารางแฮชแทน เรียกอีกอย่างว่า การทำ Rehash เพื่อที่จะพัฒนา hash table

สำหรับการทำ separate chaining จะต้องมีการกำหนดโครงสร้าง struct ดังนี้

```
struct node {  
    int info;  
    struct node *next;  
};
```

```
struct hash {  
    struct *node head;  
} hashtable[100];
```

โครงสร้าง struct node จะประกอบไปด้วย 2 필ด์คือ info สำหรับเก็บข้อมูล และ *next สำหรับเก็บแอดเดรสถัดไป ส่วนโครงสร้าง struct hash จะทำการสร้างอาร์เรย์ของพอยน์เตอร์ขนาด 100 ตำแหน่งเพื่อเก็บแอดเดรสเริ่มต้นของแต่ละตำแหน่งใน hash

การทำ Rehash

ใช้หลักการคำนวณหาตำแหน่งใหม่ หากพบว่า $h(X)$ เกิดการชนกัน ซึ่งมีหลายวิธีแต่วิธีที่ง่ายและนิยมใช้มากที่สุด คือการทำ **linear probing** โดยแนวคิดจะทดสอบตำแหน่งแอดเดรสว่างไปเรื่อยๆ จนกระทั่งพบแล้วถึงนำคีย์ข้อมูลไปเก็บไว้

$$H_{i+1}(x) = (h_i(x) + i) \bmod n$$

โดยที่ $i = 1, 2, 3, \dots$ หมายถึงรอบในการทดสอบ

$h_i(x)$ คือตำแหน่งแอดเดรสที่ชนกันก่อนหน้านี้ และ

n คือขนาดของตาราง hash

- จากสมการด้านบนจะเห็นได้ว่า ตำแหน่งแอดเดรสใหม่ $h_{i+1}(x)$ จะถูกคำนวณโดยนำตำแหน่งแอดเดรสเดิมที่มีการชนกันในรอบที่ i $h_i(x)$ ไปคำนวณรวมด้วย

เทคนิคการทำ Rehash แบบต่างๆ

- Random probing
- Quadratic probing
- Double hashing

Random probing

คำนวณตำแหน่ง address ใหม่ โดยการบวกค่า address เดิมที่ชนกับค่าคงที่ C ขึ้นมา ก่อนนำไป mod กับขนาดของ hash table

$$H_{i+1}(k) = [h_i(x) + c] \bmod n$$

Quadratic probing

คำนวณตำแหน่ง address ใหม่ โดยการบวกค่า address เดิมที่ชนกับรอบทดสอบก่อน
หน้ายกกำลังสอง ก่อนนำไป mod กับขนาดของ hash table

$$H_{i+1}(k) = [h_i(x) + i^2] \bmod n$$

Double hashing

คำนวณตำแหน่งแอดเดรสใหม่ โดยการใช้สองฟังก์ชันแฮชที่แตกต่างกัน ก่อนนำไป mod กับขนาดของ hash table

$$H_{i+1}(k) = [h_1(x) + ih_2(x)] \bmod n$$

โครงสร้างข้อมูลไพโรอริตี้คิว (priority queue)

โครงสร้างข้อมูลไพโรอริตี้คิว (priority queue)

- โครงสร้างข้อมูลไพโรอริตี้คิว (priority queue)
 - คิวที่เราสามารถเข้าถึงสมาชิกได้ตามความสำคัญของตัวสมาชิกนั้น
 - ตัวอย่างของ priority queue เช่น
 - คิวของคนไข้ในโรงพยาบาล ถ้ามีคนไข้หกรอกวอยู่ แต่มีคนไข้เลือดฟุ้งไม่หยุดมาที่หลัง คนไข้เลือดฟุ้งไม่หยุดก็ควรได้รับการรักษาก่อน ไม่เช่นนั้นก็ตายแน่นอน
 - คิวของการใช้เครื่องพิมพ์ ซึ่งคนที่พิมพ์จำนวนหน้าน้อยควรได้รับการพิจารณาก่อน
- โดยทั่วไปถ้าเป็นคิวที่สมาชิกเทียบค่ากันได้นั้น จะให้ค่าน้อยถือว่าสำคัญกว่าค่ามาก และถ้าสมาชิกสองตัวมีค่าเท่ากัน ตัวที่อยู่ในคิวมานานกว่าก็ควรจะถือว่าสำคัญกว่า

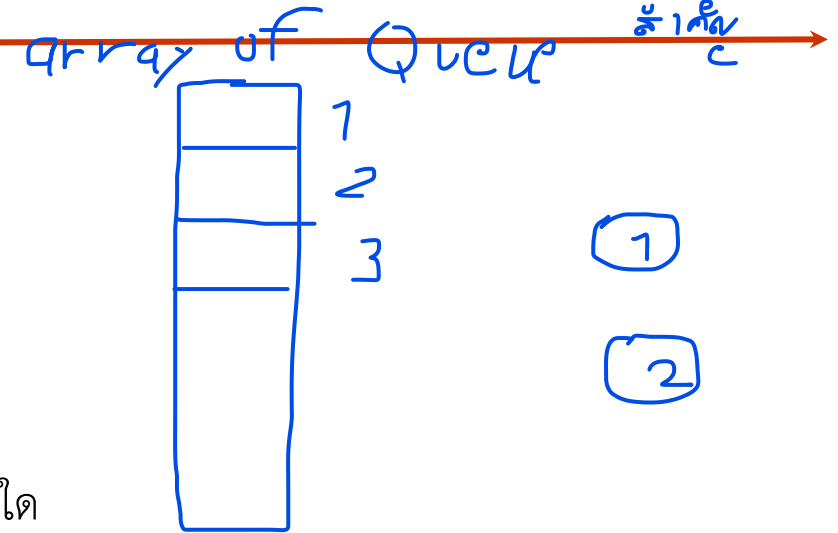
โครงสร้างข้อมูลไพโรอริตี้คิว (priority queue)

Course	Priority	Due day
Database Systems	2	October 3
UNIX	4	October 10
Data Structure & Algorithm	1	September 29
Structured Systems Analysis	3	October 7

การสร้าง priority queue

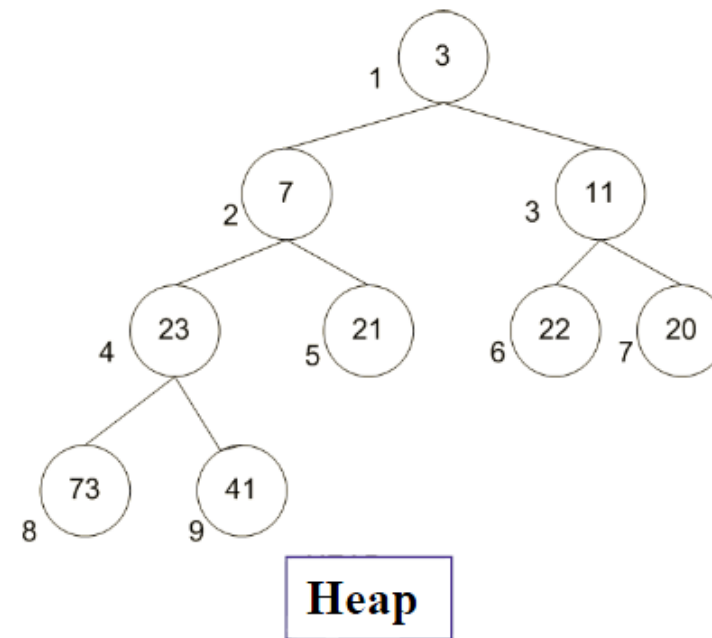
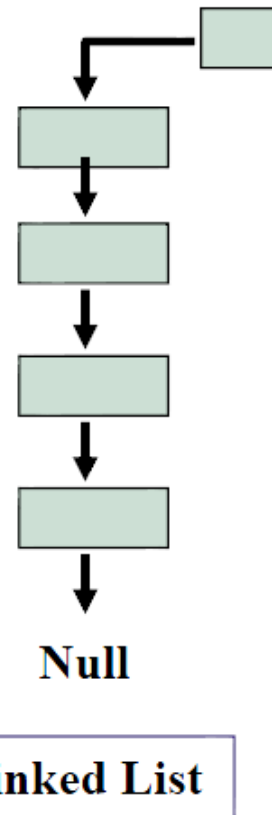
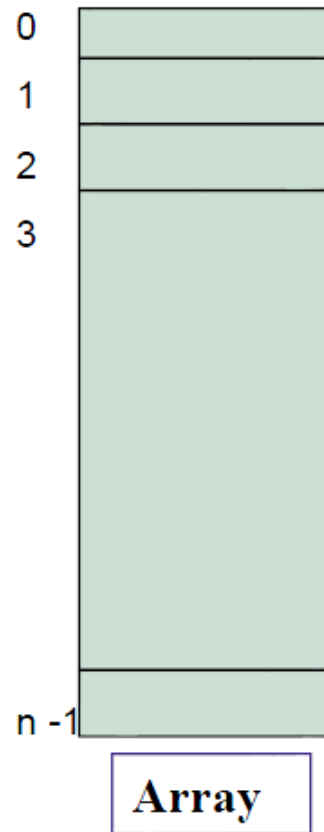
จัดลำดับความ

สำคัญ

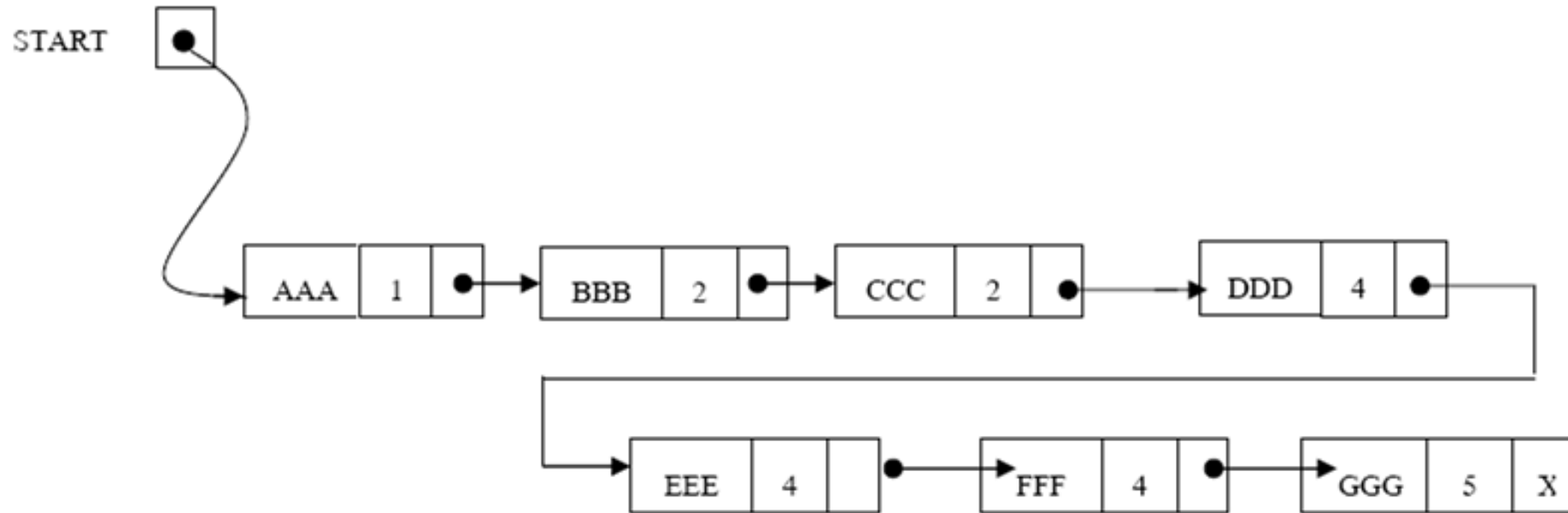


- ใช้คิวธรรมดา ทำไม่ได้เพราะไม่มีการจัดลำดับความสำคัญของสมาชิก
- ใช้อาร์เรย์ของคิว คือมีแต่ละช่องเป็นลิสต์ที่เก็บของในแต่ละ priority ไว้
 - วิธีนี้มีข้อเสียคือ ถ้ามี priority จำนวนมากจะทำให้เปลืองที่มาก
 - ไม่รู้อีกด้วยว่าจะมีกี่ priority ดังนั้นจึงไม่รู้ว่าจะต้องสร้างอาร์เรย์ขนาดใหญ่เท่าใด
- ใช้อาร์เรย์ลิสต์ การใช้วิธีนี้ทำให้เรียงสมาชิกได้ง่าย getMin สามารถทำได้ด้วยเวลาคงที่ แต่ตอนเต็มจะเสียเวลาหาตำแหน่งที่เต็ม และเสียเวลาเลื่อนอาร์เรย์ตอนเอาของออกด้วย (อาจมีการเลื่อนช่องเยอะ) เวลาที่เสียนั้นเป็นเวลาเชิงเส้น แต่ก็ยังถือว่าเข้าไปอยู่ดี
- ใช้ Linked List วิธีนี้ตอนที่เอาของออกจะไม่ต้องทำอะไรเพิ่ม แค่ update pointer เพราะฉะนั้นเวลาในการ removeMin จะคงที่ แต่ตอนเต็มของลงในคิวก็ต้องใช้เวลาหาช่องเต็ม

การสร้าง priority queue

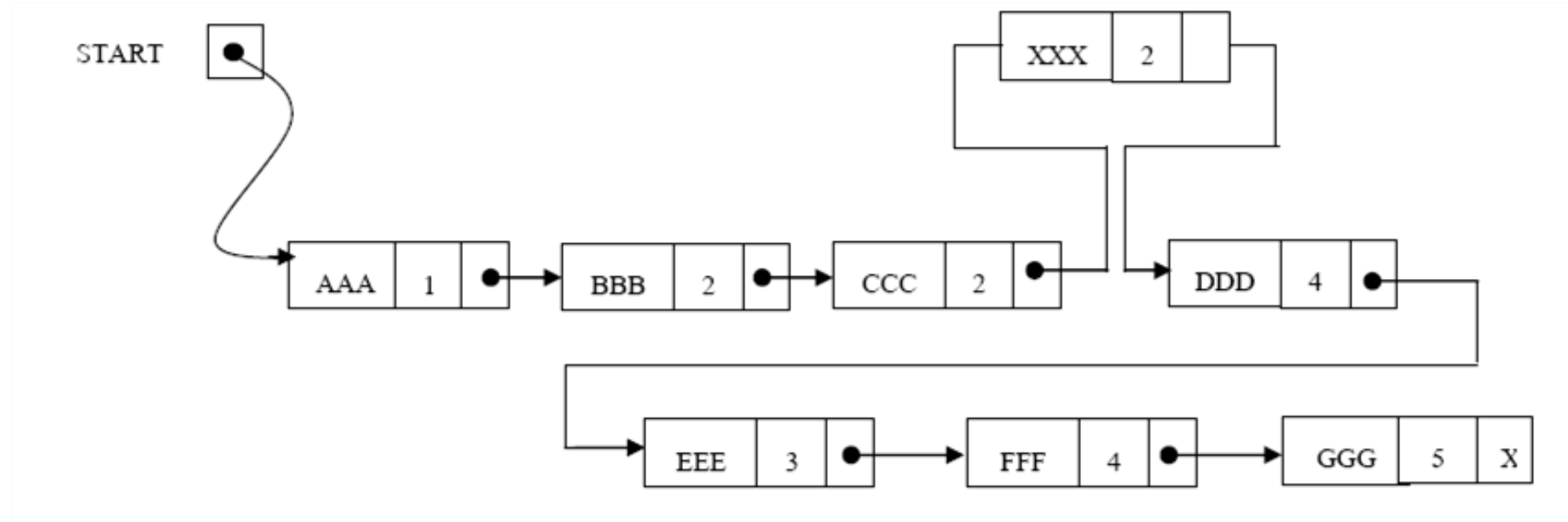


การสร้าง priority queue



การเก็บแบบรายการโยง

การสร้าง priority queue



การแทนแวลำดับด้วยคิวจัดอันดับ

การสร้าง priority queue

	FRONT	REAR		1	2	3	4	5	6
1	2	2	1		AAA				
2	1	3	2	BBB	CCC	XXX			
3	0	0	3						
4	5	1	4	FFF				DDD	EEE
5	4	4	5				GGG		

การเก็บคิวในแถวลำดับ

โครงสร้างข้อมูลไพร์ออริตี้คิว (priority queue)

- priority queue มีฟังก์ชันหลัก ๆ สำหรับทำงานดังนี้
 - `insert(item i)` (enqueue) เป็นการเพิ่มสมาชิกใหม่ในคิว
 - `delete()` (dequeue) ลบสมาชิกที่มีความสำคัญมากที่สุดออกไปจากคิว
 - `find()` หาสมาชิกที่มีความสำคัญมากที่สุด(highest priority)
 - `decreasePriority(item i, p)` ลดความสำคัญของสมาชิกตัวที่ i ด้วยค่า p
- ใน priority queue หลักการของ "first in first out" ไม่สามารถเอามาใช้ได้.

สมมติฐานของ Priority Queues

- ความสำคัญมากที่สุด(the highest priority) สามารถพิจารณาได้จากค่า *minimum* ของข้อมูล หรือค่า *maximum* ของข้อมูล
 - เช่น highest priority คือค่า *minimum*.
 - จะเรียกฟังก์ชันของการดึงข้อมูลคือ *deleteMin()*.
 - เรียกฟังก์ชันของการค้นหาข้อมูลคือ *findMin()*.

การสร้าง priority queue (รายการไม่เรียงลำดับ: Array)

1. ใช้อาร์เรย์ *arr*.

- *insert()* เพิ่มข้อมูลตัวใหม่ไปยังตำแหน่งว่างถัดไปใน *arr*, ใช้เวลา $\Theta(1)$.
- *findMin()* เวลาที่แย่ที่สุดคือ $\Theta(n)$
- *deleteMin()* เวลาที่แย่ที่สุดคือ $\Theta(n)$
 - $\Theta(n)$ สำหรับหาค่าที่น้อยที่สุด
 - $\Theta(1)$ เอาสมาชิกท้ายสุดไปยังตำแหน่งที่ถูกลบ
- *DecreasePriority(i, p)* – ลด priority ของสมาชิกตัวที่ *i* ที่อยู่ในอาร์เรย์ *arr[i]* เป็น *p* ด้วย $\Theta(1)$

4,x	8,a	1,b	9,c	7,y	
0	1	2	3	4	5

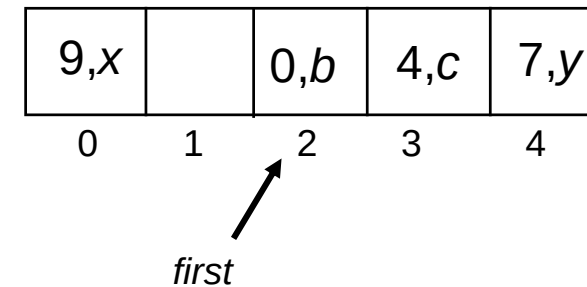
2. ใช้ linked list.

- *insert()* ใช้เวลา $\Theta(1)$
- *findMin()* ใช้เวลา $\Theta(n)$ เพราะต้องค้นหาทั้งลิสต์
- *deleteMin()* ใช้เวลา $\Theta(n)$
 - ค้นหาทั้งลิสต์ใช้เวลา $\Theta(n)$
 - ลบข้อมูลใช้เวลา $\Theta(1)$

การสร้าง priority queue (รายการเรียงลำดับ : Circular Array)

1. ใช้อาร์เรย์ A.

- *insert()* ต้องให้อาร์เรย์เรียงลำดับข้อมูลด้วย.
 - เวลาที่แย่ที่สุดคือ $\Theta(n)$
 - สมาชิกตัวใหม่ที่ป้อนเข้าไปต้องป้อนหลังข้อมูลที่มี highest priority.
ดังนั้น ต้องย้าย $n-1$ สมาชิก
- *findMin()* ใช้เวลา $\Theta(1)$
- *deleteMin()* ใช้เวลา $\Theta(1)$
- *DecreasePriority(i, p)* – ลด priority ของสมาชิกตัวที่ i / แล้ว reinsert ด้วยเวลา $\Theta(n)$



2. ใช้ linked list.

- *insert()* ใช้เวลา $\Theta(n)$
 - กรณีแย่ที่สุดคือต้องค้นหาทั้งลิสต์เพื่อหาตำแหน่งที่จะแทรก
- *findMin()* ใช้เวลา $\Theta(1)$
- *deleteMin* ใช้เวลา $\Theta(1)$

การวิเคราะห์ประสิทธิภาพ

โครงสร้างข้อมูล	Insert –worst case	deleteMin-worst case
Heap	$\Theta(\lg n)$	$\Theta(\lg n)$
Unsorted (array or linked list)	$\Theta(1)$	$\Theta(n)$
Sorted (array or linked list)	$\Theta(n)$	$\Theta(1)$



การใช้ลิงค์ลิสต์สร้าง priority queue

```
struct node
{
    int priority;
    int info;
    struct node *link;
}*front = NULL;
```

การใช้ลิงค์ลิสต์สร้าง priority queue

```
insert()  
{   struct node *tmp,*q;  
    int added_item,item_priority;  
    tmp = (struct node *)malloc(sizeof(struct node));  
    printf("Input the item value to be added in the queue : ");  
    scanf("%d",&added_item);  
    printf("Enter its priority : ");  
    scanf("%d",&item_priority);  
    tmp->info = added_item;  
    tmp->priority = item_priority;  
    /*Queue is empty or item to be added has priority more than first  
    item*/
```



การใช้ลิงค์ลิสต์สร้าง priority queue

```
if( front == NULL || item_priority < front->priority )
{
    tmp->link = front;
    front = tmp;
}
else
{
    q = front;
    while( q->link != NULL && q->link->priority <= item_priority )
        q=q->link;
    tmp->link = q->link;
    q->link = tmp;
}/*End of else*/
}/*End of insert()*/
```



การใช้ลิงค์ลิสต์สร้าง priority queue

```
del()
{
    struct node *tmp;
    if(front == NULL)
        printf("Queue Underflow\n");
    else
    {
        tmp = front;
        printf("Deleted item is %d\n", tmp->info);
        front = front->link;
        free(tmp);
    }
} /*End of del() */
```



การใช้ลิสต์สร้าง priority queue

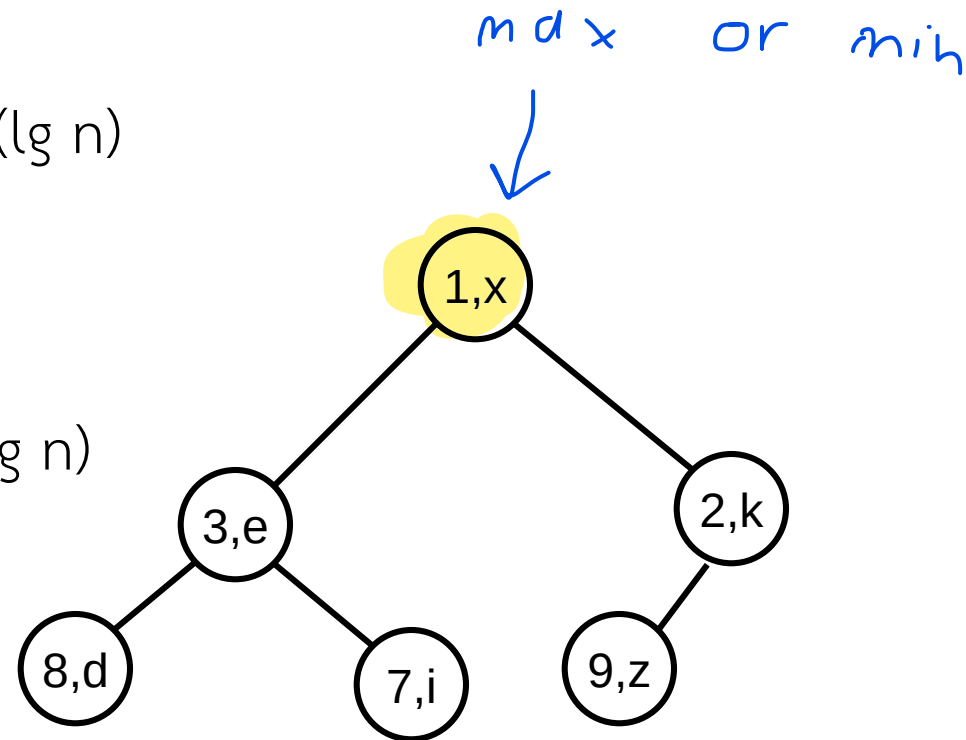
```
display()
{
    struct node *ptr;
    ptr = front;
    if(front == NULL)
        printf("Queue is empty\n");
    else
    {
        printf("Queue is :\n");
        printf("Priority      Item\n");
        while(ptr != NULL)
        {
            printf("%5d      %5d\n",ptr->priority,ptr->info);
            ptr = ptr->link;
        }
    }
    /*End of else */
}/*End of display() */
```

ฮีป(Heap)

- นำหลักการของต้นไม้บริบูรณ์มาสร้าง priority queue
- ต้นไม้บริบูรณ์ (complete binary tree) คือต้นไม้ที่เต็มทุกระดับ ยกเว้นระดับของใบ ซึ่งระดับของใบนี้จะต้องถูกเติมจากซ้ายไปขวาเท่านั้น จะไม่มีรูว่างเป็นหย่อมๆ
- ฮีปคือต้นไม้บริบูรณ์
 - กรณีสมาชิกที่อยู่ที่รากมีค่าน้อยที่สุด(เราถือว่าค่าน้อยสำคัญกว่าค่ามาก ซึ่งฮีปที่เรียงของแบบนี้ เราเรียกว่า minheap)
 - ต้นไม้ย่อยทั้งต้นซ้ายและขวา ก็เป็นฮีปเช่นเดียวกันโปรดสังเกต ว่าฮีปนั้นไม่ใช่ต้นไม้ค้นหาแบบทวิภาค(binary search tree) เพราะอย่างไรก็ตามค่าที่น้อยที่สุดก็อยู่ที่รากของมัน จะเห็นว่าค่าน้อยเสมอไม่ว่าจะเป็นต้นไม้ย่อยใดๆ

ฮีป(Heap)

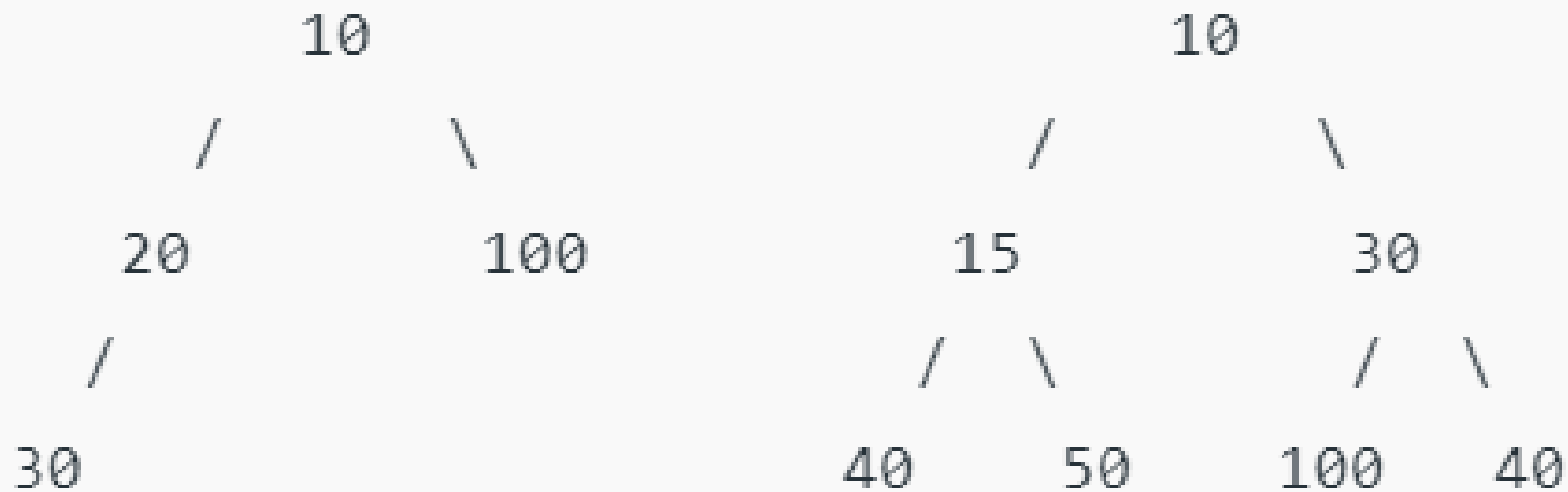
- Heap.
 - กรณีแย่งที่สุด $insert()$ ใช้เวลา $\Theta(\lg n)$
 - $deleteMin()$ ใช้เวลา $\Theta(\lg n)$
 - $findMin()$ ใช้เวลา $\Theta(1)$
 - $decreaseKey(i, p)$ ใช้เวลา $\Theta(\lg n)$



Binary Heap

- A Binary Heap is a Binary Tree with following properties.
- 1) It's a complete tree (All levels are completely filled except possibly the last level and the last level has all keys as left as possible). This property of Binary Heap makes them suitable to be stored in an array.
- 2) A Binary Heap is either Min Heap or Max Heap. In a Min Binary Heap, the key at root must be minimum among all keys present in Binary Heap. The same property must be recursively true for all nodes in Binary Tree. Max Binary Heap is similar to MinHeap.

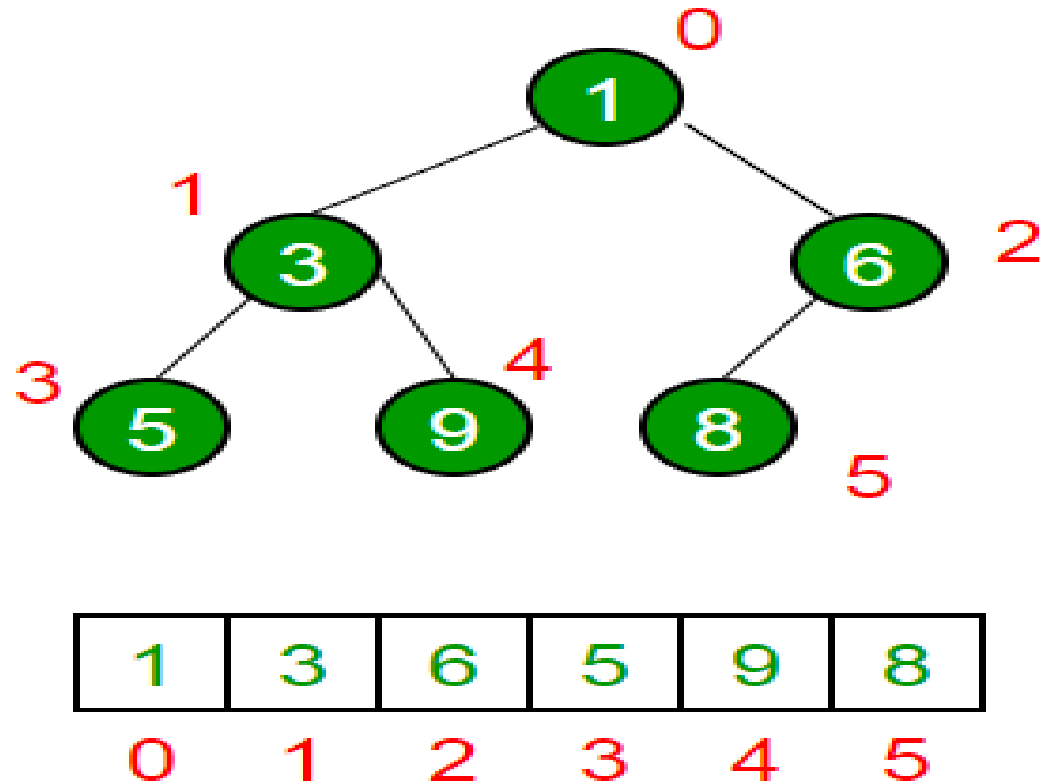
Examples of Min Heap:



How is Binary Heap represented?

- A Binary Heap is a Complete Binary Tree. A binary heap is typically represented as an array.
- The root element will be at $\text{Arr}[0]$.
- Below table shows indexes of other nodes for the i th node, i.e., $\text{Arr}[i]$:
 - $\text{Arr}[(i-1)/2]$ Returns the parent node
 - $\text{Arr}[(2*i)+1]$ Returns the left child node
 - $\text{Arr}[(2*i)+2]$ Returns the right child node

The traversal method use to achieve Array representation is **Level Order**



Applications of Heaps:

- 1) Heap Sort: Heap Sort uses Binary Heap to sort an array in $O(n \log n)$ time.
- 2) Priority Queue: Priority queues can be efficiently implemented using Binary Heap because it supports `insert()`, `delete()` and `extractmax()`, `decreaseKey()` operations in $O(\log n)$ time. Binomial Heap and Fibonacci Heap are variations of Binary Heap. These variations perform union also efficiently.
- 3) Graph Algorithms: The priority queues are especially used in Graph Algorithms like Dijkstra's Shortest Path and Prim's Minimum Spanning Tree.
- 4) Many problems can be efficiently solved using Heaps. See following for example.
 - a) K'th Largest Element in an array.
 - b) Sort an almost sorted array/
 - c) Merge K Sorted Arrays.

Operations on Min Heap:

- 1) `getMini()`: It returns the root element of Min Heap. Time Complexity of this operation is $O(1)$.
- 2) `extractMin()`: Removes the minimum element from MinHeap. Time Complexity of this Operation is $O(\text{Log}n)$ as this operation needs to maintain the heap property (by calling `heapify()`) after removing root.
- 3) `decreaseKey()`: Decreases value of key. The time complexity of this operation is $O(\text{Log}n)$. If the decreases key value of a node is greater than the parent of the node, then we don't need to do anything. Otherwise, we need to traverse up to fix the violated heap property.
- 4) `insert()`: Inserting a new key takes $O(\text{Log}n)$ time. We add a new key at the end of the tree. IF new key is greater than its parent, then we don't need to do anything. Otherwise, we need to traverse up to fix the violated heap property.
- 5) `delete()`: Deleting a key also takes $O(\text{Log}n)$ time. We replace the key to be deleted with minus infinite by calling `decreaseKey()`. After `decreaseKey()`, the minus infinite value must reach root, so we call `extractMin()` to remove the key.